UNION

- **Definition**

The UNION operator is used to combine the result-set of two or more SELECT statements. The UNION operator automatically **removes duplicate rows** from the result set.

- **Syntax**

  SELECT column_name(s) FROM table1
  UNION
  SELECT column_name(s) FROM table2;

- **Rules & Conditions**

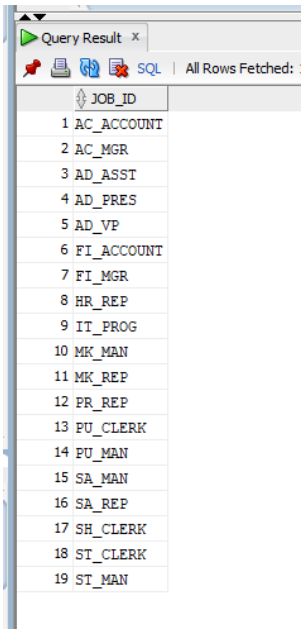
  The UNION function combines the results of two or more SELECT queries into a single result set, removing duplicate rows. Each SELECT statement within the UNION must have the same number of columns. Also, they have to have similar data types, and the columns must also be in the same order.

- **Difference between UNION and UNION ALL**

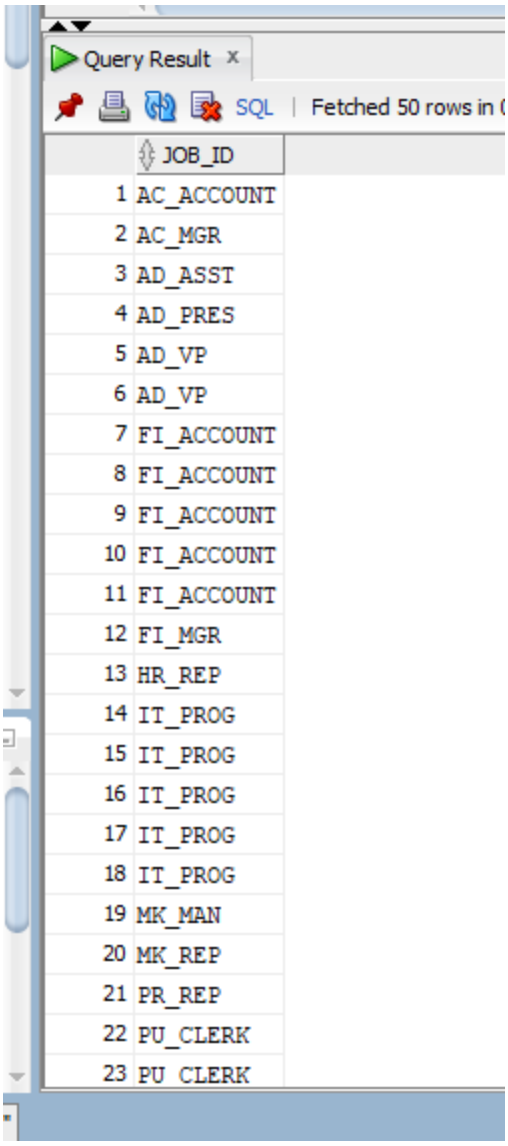
UNION	UNION ALL
Removes duplicate rows, returning only unique records.	Includes all rows, even duplicates, from both result sets.
Slower due to the need to eliminate duplicates.	Faster, as it doesn't check for duplicates and simply combines the rows.
The result set is smaller as duplicates are removed.	The result set is larger because duplicates are included.
UNION removes duplicates internally (via sorting or hashing), but the final result order is not guaranteed unless ORDER BY is specified.	No sorting is done unless explicitly mentioned using ORDER BY
Use when you need distinct results in the final output.	Use when retaining all rows, including duplicates, is important.
Requires more processing power because of the duplicate elimination process.	More efficient as no additional computation is needed to remove duplicates.
NULLs are treated as duplicates if they appear in both result sets.	NULLs are treated as regular values and included in the result, even if repeated.

Works well for queries where distinct data is crucial, like in reporting.	Suitable for queries where duplicates are not problematic, such as transaction logs or detailed data analysis.
Can be used in cases where you want to enforce uniqueness across different datasets.	Useful for combining large datasets where duplicate entries represent meaningful records, like in data aggregation.
SQL engine performs a distinct operation internally to remove duplicates.	No distinct operation is performed; all rows, including exact duplicates, are included.
Suitable when result accuracy is more important than performance.	Preferred when performance is the priority and duplicates don't interfere with analysis or data quality.

- **Practical Example** – Write a simple example query for each operation using any relevant tables from the HR schema or the Company schema.
  SELECT job_id FROM employees UNION SELECT job_id FROM job_history;



SELECT job_id FROM employees UNION ALL SELECT job_id FROM job_history;



	JOB_ID
1	AC_ACCOUNT
2	AC_MGR
3	AD_ASST
4	AD_PRES
5	AD_VP
6	AD_VP
7	FI_ACCOUNT
8	FI_ACCOUNT
9	FI_ACCOUNT
10	FI_ACCOUNT
11	FI_ACCOUNT
12	FI_MGR
13	HR_REP
14	IT_PROG
15	IT_PROG
16	IT_PROG
17	IT_PROG
18	IT_PROG
19	MK_MAN
20	MK_REP
21	PR_REP
22	PU_CLERK
23	PU_CLERK

INTERSECT

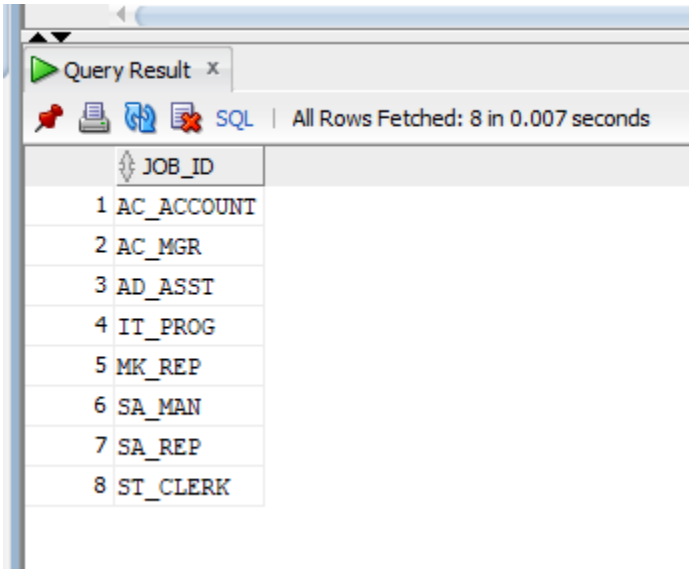
- **Definition**
  An INTERSECT combines two or more queries into a single query and returns only data that is found in the result set of both individual queries. The result set does not contain any duplicates.

- **Syntax**
  SELECT column1, column2,..., columnN FROM table1, table2,..., tableN
  INTERSECT
  SELECT column1, column2,..., columnN FROM table1, table2,..., tableN

- **Rules & Conditions**
  Both SELECT statements must return the exact same number of columns.

- **Practical Example**

  SELECT job_id FROM employees
  INTERSECT
  SELECT job_id FROM job_history;



EXCEPT

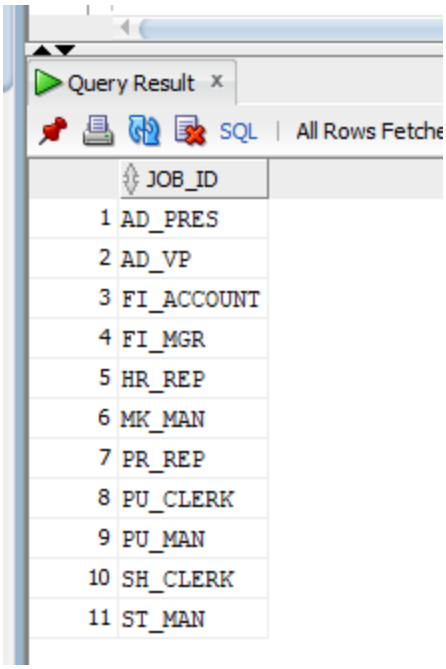
- **Definition**
  An EXCEPT combines two or more queries into a single query and returns data that is found in the result set of the first query, but not of the second query. The result set can be thought of as the difference between the first query and the second query.

- **Syntax**
  SELECT column1, column2,..., columnN FROM table1, table2,..., tableN
  EXCEPT
  SELECT column1, column2,..., columnN FROM table1, table2,..., tableN

- **Rules & Conditions**
  Both SELECT statements must have the exact same number of columns or expressions. The columns must be in the same logical order in both queries. The corresponding columns in both queries must have the same or highly compatible data types.

- **Practical Example**

  SELECT job_id FROM employees
  MINUS
  SELECT job_id FROM job_history;



	JOB_ID
1	AD_PRES
2	AD_VP
3	FI_ACCOUNT
4	FI_MGR
5	HR_REP
6	MK_MAN
7	PR_REP
8	PU_CLERK
9	PU_MAN
10	SH_CLERK
11	ST_MAN